

Task 6: Multi Document Agents — A 3-Agent RAG Platform with OCR and Voice Support

September 4, 2026

Project Overview

Multi Document Agents is a multi-agent Retrieval-Augmented Generation (RAG) platform that lets users chat with their documents through three specialized agents — **Retriever**, **Analyst**, and **Answer** — coordinated by a LangGraph orchestrator, instead of relying on a single model to do everything. Uploaded documents (PDF, TXT, MD, PY, YAML, JSON, CSV, DOCX, PPTX) are cleaned, chunked, embedded, and stored in a vector database. The platform additionally supports OCR (image-to-text) and voice transcription (speech-to-text, Arabic/English) as extra input modalities, so users can ask questions using text, images, or audio. A FastAPI backend exposes all functionality through a REST API, consumed by a Streamlit front end, with LangSmith used to monitor and trace every agent run.

Division of Responsibilities

File	Salma Muhammed Entsar Functionality
src/backend/main.py	FastAPI application exposing the full REST API: document upload & indexing (/documents/upload), text chat (/chat), OCR (/ocr), audio transcription (/transcribe), and multimodal chat (/chat/multimodal); validates supported file types, handles uploads safely with UUID-prefixed filenames, and wires all services (ingestion pipeline, OCR, Whisper) into the API layer.
src/ui/app.py	Streamlit user interface with tabs for document upload, text chat, image OCR, audio transcription, and multimodal chat; manages session state (thread ID, chat history, OCR and transcription text) and calls the FastAPI backend for every action.
src/additional_features_task2/ocr/ocr_feature.py	OCRService: wraps an OCR engine (EasyOCR) to extract text from uploaded images (PNG, JPG, JPEG, WEBP) for use as chat context.
src/additional_features_task2/voice/voice_feature.py	WhisperService: transcribes uploaded audio files (WAV, MP3, M4A, FLAC, OGG) into text in Arabic or English for use as chat context.
src/ingestion/loader.py	Loads raw content from uploaded files into LangChain Document objects.
src/ingestion/cleaner.py	Cleans and normalizes raw document text before chunking.
src/ingestion/chunker.py	Splits cleaned documents into smaller, overlapping text chunks suitable for embedding and retrieval.
src/ingestion/embedder.py	Loads and wraps the embedding model used to convert text chunks into vector representations.
src/ingestion/pipeline.py	IngestionPipeline: orchestrates the full ingestion process (load → clean → chunk → embed → store) and returns the number of chunks created per document.
src/retrieval/semantic_search.py	Embeds the user query and retrieves document chunks by cosine similarity against stored vectors.
src/retrieval/keyword_search.py	Retrieves chunks via lexical/exact-term matching (e.g. BM25), complementing semantic search for technical terms, IDs, and equations.
src/retrieval/hybrid_search.py	Combines semantic and keyword search results into a single ranked candidate set.
src/retrieval/metadata_filter.py	Restricts retrieval to chunks matching specific metadata (document name, page, chapter, author, date, type).
src/retrieval/reranker.py	Re-scores and reorders initial candidate chunks with a more accurate relevance model before selection.
src/retrieval/context_selector.py	Selects a compact, diverse, high-quality subset of reranked chunks (e.g. best 5–8) to pass to the Analyst Agent, preserving citation metadata.
src/retrieval/vector_store.py	Wraps the vector database; stores and persists embedded chunks and exposes similarity search.
src/agents/retriever.py	Retriever Agent: searches the vector database for relevant evidence using the tools above (query rewriting, semantic/keyword/hybrid search, metadata filtering, reranking, context selection) without attempting to answer the question itself.
src/config/prompts/retriever.py	System prompt defining the Retriever Agent’s role, constraints, and tool-use instructions.
LangSmith configuration	Sets up tracing/monitoring so every agent and tool invocation across the workflow can be inspected and debugged in LangSmith.

Table 1: Files and functionality — Salma Muhammed Entsar.

File	Maya Ashraf Functionality
<code>src/agents/analyst.py</code>	Analyst Agent: evaluates the evidence gathered by the Retriever, compares facts, extracts tables, and performs calculations; triggers a feedback loop back to the Retriever Agent when it detects missing information.
<code>src/config/prompts/analyst.py</code>	System prompt defining the Analyst Agent’s role, evaluation criteria, and feedback-loop instructions.
<code>src/agents/answer.py</code>	Answer Agent: once the Analyst confirms there is sufficient evidence, crafts the final response — clear text or tables with precise page and document citations.
<code>src/graph/state.py</code>	Defines the shared workflow state schema (query, retrieved evidence, analysis results, answer, feedback flags, iteration tracking) passed between agents across the graph.
<code>src/graph/nodes.py</code>	Defines the individual LangGraph nodes wrapping each agent (Retriever, Analyst, Answer) as callable graph steps.
<code>src/graph/edges.py</code>	Defines the conditional edges/routing logic of the graph, including the Analyst → Retriever feedback loop and the path to the Answer Agent once evidence is sufficient.
<code>src/graph/workflow.py</code>	run_graph: builds and compiles the full Orchestrator → Retriever → Analyst → Answer LangGraph workflow (with the Analyst–Retriever feedback loop) and exposes the entry point used by both the text-chat and multimodal-chat API endpoints.
<code>src/tools/calculator.py</code>	Performs accurate numerical calculations for the Analyst Agent (sums, averages, percentages, ratios, statistical comparisons) instead of relying on the LLM’s internal arithmetic.
<code>src/tools/table_extractor.py</code>	Identifies and extracts structured tables from documents (e.g. model performance metrics) into rows/columns the Analyst Agent can process.
<code>src/tools/document_comparator.py</code>	Compares information across multiple documents — similarities, differences, methodologies, results — to support cross-document questions.
<code>src/tools/data_analysis.py</code>	Performs structured analysis over numerical/tabular evidence: averages, percentages, rankings, distributions, and trend identification.

Table 2: Files and functionality — Maya Ashraf.